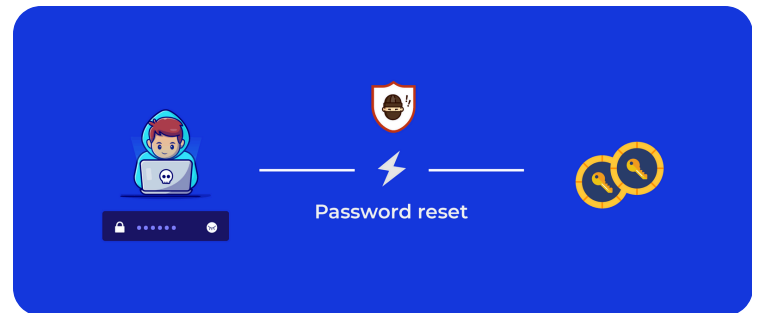




July 13, 2024

How to Implement a Forgot Password Flow? Complete Guide

What should happen on the backend when a user forgets their password? Read to find a pseudo code implementation of the simplest way to reset passwords securely.



JC

By Joel Coutinho
Software Developer at SuperTokens

Table of contents:

1. [Implementing a Secure Forgot Password Flow](#)
2. [Best Practices for Password Reset Flow](#)
3. [3 Common Issues in Implementing Forgot Password Flow](#)
4. [Conclusion](#)



Step 1: User Enters Their Email

When users forget their passwords, they start the password reset process by entering their email address into a form. This form should be simple and clear, guiding users to input the email address associated with their account.

Step 2: Create a Password Reset Token

Once the email is submitted, the backend checks if the email exists in the database. Regardless of whether the email exists or not, the system should display a message stating that an email has been sent. This prevents attackers from determining if an email address is valid.

If the email exists, a new password reset token is generated. Using a secure random source, the token should be a long (at least 64 characters) random string to prevent brute force attacks.

Step 3: Store the Token in the Database

The generated token is then hashed using SHA-256 and stored in the database along with the user's ID and an expiration time. This ensures that the token is only valid for a limited period, mitigating the risk of attacks using expired tokens. Here's an example schema for storing password reset tokens:

```
CREATE TABLE password_reset_tokens (  
  user_id VARCHAR(36) NOT NULL,  
  token VARCHAR(128) NOT NULL UNIQUE,  
  token_expiry BIGINT UNSIGNED NOT NULL,  
  PRIMARY KEY (user_id, token)  
);
```



Step 4: Send the Token to the User



<https://example.com/reset-password?token=<Token here>>



Step 5: Verify the Token and Update the Password

When the user clicks the link and submits the new password, the backend verifies the token. If the token's hash matches the one stored in the database and it hasn't expired, the user's password is updated. Here's a pseudo code example of the backend logic:

```
function redeemToken(passwordResetToken, newPassword) {
  hashedToken = hash_sha256(passwordResetToken);
  rowFromDb = db.getRowTheContains(hashedToken);
  if (rowFromDb == null) {
    throw Error("Invalid password reset token");
  }
  userId = rowFromDb.user_id;

  db_startTransaction(() => {
    allTokensForUser = db_getAllTokensBasedOnUser(userId);

    matchedRow = allTokensForUser.find(row => row.token == hashedToken);
    if (matchedRow == null) {
      throw Error("Invalid password reset token");
    }

    if (matchedRow.token_expiry < time_now()) {
      db_rollback();
      throw Error("Token has expired. Please try again");
    }

    db_deleteAllRowsForUser(userId);

    hashedAndSaltedPassword = hashAndSaltPassword(newPassword);
    db_saveNewPassword(userId, hashedAndSaltedPassword);
    db_commitTransaction();
  });
}
```





-
- **Use Secure Random Tokens:** Ensure tokens are generated from a secure random source and are sufficiently long (≥ 64 characters) to prevent brute force attacks.
 - **Hash Tokens in the Database:** Store only the hashed version of tokens in the database to protect against database theft.
 - **Expire Tokens:** Set a reasonable expiration time for tokens to limit the window of opportunity for an attack.
 - **Implement Rate Limiting:** To prevent abuse, limit the number of password reset requests that can be made within a given time frame.
 - **Secure Token Delivery:** Send tokens via secure email channels and consider using additional layers of authentication, such as two-factor authentication.

Common Issues in Implementing Forgot Password Flow

Issue #1: Brute Force Attacks

This is a common threat for all web applications. Attackers may attempt to detect patterns in the password reset tokens - like if it's derived from a user's userId, time they signed up, their email or any other information. Attackers may also try all possible combinations of letters and numbers (brute force), and may even succeed if the generated tokens are not long or random enough (i.e. have low entropy).

Solution: Generate tokens using a secure random source and ensure they are long enough to be unguessable. Implement rate limiting to prevent repeated attempts.

Issue #2: Database Theft of Tokens



While there are plenty of problems with an attacker getting database access, one of them is their ability to get users' password reset tokens, like in this research on Paleohacks. To mitigate this risk, we store only the hashed version of tokens in the database. Passwords are hashed and stored and it's important that password reset tokens are too (for the same reasons).

Another related attack vector is the use of JWTs as the password reset token. Whilst this makes development easy, a major risk is that if the secret key used to sign them is compromised, the attacker can use that to generate their own valid JWT. This would allow them to reset any user's password. The JWT secret key (or signing key) must be carefully protected and hence we do not recommend using JWTs as the password reset token.

Issue #3: Reusing Existing Tokens

For simplicity of development, it may be tempting to store a "static" password reset token per user. This token might be randomly generated on user sign up, or based on their password's hash or some other information.

This in turn implies that these tokens cannot be stored in a hashed form in the database - since we will need to send this token over email (because if we hash it, we can't unhash it at the time). Therefore, if the database is compromised, these tokens can be used to reset a user's password.

Another risk of reusing tokens is that if a token somehow gets leaked, even if it has been redeemed by the actual user, it can still be used by an attacker to change that user's password.

What actually happens on the backend (pseudo code)

Applications need to account for the frequency with which users forget their passwords. This opens a potential attack vector because anyone can request a new password on behalf of the legitimate user. Resetting a password requires sending a token to a user's email address and this



Security Issues to Consider

Brute force attacks

This is a common threat for all web applications. Attackers may attempt to detect patterns in the password reset tokens - like if it's derived from a user's userId, time they signed up, their email or any other information. Attackers may also try all possible combinations of letters and numbers (brute force), and may even succeed if the generated tokens are not long or random enough (i.e. have low entropy).

To prevent this, we must ensure that tokens are generated using a secure random source, and that they are long enough (we recommend ≥ 64 characters). Later on in this blog, we will see one such method.

Database theft of password reset tokens

There are several ways an attacker can gain access to an application's database: SQL injection attacks, targeting unpatched database vulnerabilities, and exploiting unused database services. They could even gain access if someone hasn't updated the default login credentials.

While there are plenty of problems with an attacker getting database access, one of them is their ability to get users' password reset tokens, like in this research on [Paleohacks](#). To mitigate this risk, we store only the hashed version of tokens in the database. Passwords are hashed and stored and it's important that password reset tokens are too (for the same reasons).

Another related attack vector is the use of JWTs as the password reset token. Whilst this makes development easy, a major risk is that if the secret key used to sign them is compromised, the



Reusing existing tokens

For simplicity of development, it may be tempting to store a “static” password reset token per user. This token might be randomly generated on user sign up, or based on their password’s hash or some other information.

This in turn implies that these tokens cannot be stored in a hashed form in the database - since we will need to send this token over email (because if we hash it, we can’t unhash it at the time). Therefore, if the database is compromised, these tokens can be used to reset a user’s password.

Another risk of reusing tokens is that if a token somehow gets leaked, even if it has been redeemed by the actual user, it can still be used by an attacker to change that user’s password.

Conclusion

Implementing a secure password reset flow requires attention to detail and a thorough understanding of security best practices. By generating secure tokens, storing them safely, and using additional authentication measures, you can protect your users’ accounts from common threats. At SuperTokens, we provide easy-to-use, open-source authentication solutions to help you secure your applications and save development time.

Super



Joe Coutinho

Software Developer at SuperTokens



Software dev and content lead at SuperTokens. Contributed to the SuperTokens core, backend SDKs, and developer documentation.



Try SuperTokens in under **5 minutes**

SuperTokens provides open source user authentication that is quick to implement and easy to customize.

[Learn More](#)[Product](#)[Pricing](#)[Product Roadmap](#)[Self Hosted](#)[Features](#)[Magic Link](#)[SSO \(Single-Sign On\)](#)[Multi-Tenant](#)



guides

API References

Social Login

Passwordless Login

Email Password Login

Company

About Us

Blog

Careers

Customers

Community

Engineering

Launches

Security at SuperTokens

Resources

Support

Contribute

Contact Sales

Terms of Service

Privacy Policy

API Status

Auth0 vs SuperTokens
vs Cognito

JWT Decoder





[View Report](#)